

状压 DP

用一个整数表示集合状态，在可控的指数复杂度内完成动态规划。

UPDATED / 2026-07-30

1 什么时候考虑状压

看到集合规模很小，通常是 $n \leq 20$ ，而一个状态需要记录“哪些元素已经选择、访问或匹配”时，可以考虑用二进制位压缩集合。

令第 i 位表示第 i 个元素是否在集合中，则一共有 2^n 个状态。设计前先估算总复杂度：

- 只遍历所有状态是 $O(2^n)$ ；
- 每个状态再枚举一个元素是 $O(n2^n)$ ；
- 对每个集合枚举它的所有子集，总复杂度是 $O(3^n)$ 。

不要只看到 n 很小就立刻状压，还要确认状态是否包含了完成转移所需的全部信息。

2 集合的常见写法

下面假设 mask 、 A 和 B 都是非负整数，第 i 位对应元素 i 。

	操作 写法
判断元素 i 是否存在	$\text{mask} \gg i \ \& \ 1$
加入元素 i	$\text{mask} \ \vee \ (1 \ll i)$
删除元素 i	$\text{mask} \ \& \ \sim(1 \ll i)$
翻转元素 i	$\text{mask} \ \wedge \ (1 \ll i)$
判断 $B \subseteq A$	$(A \ \& \ B) == B$
集合交	$A \ \& \ B$
集合并	$A \ \vee \ B$
集合差 $A - B$	$A \ \& \ \sim B$
元素数量	<code>__builtin_popcount(mask)</code>

2.1 计算 $A - B$

通用写法是：

```
int rest = A & ~B;
```

如果已经确定 $B \subseteq A$ ，也可以写：

```
int rest = A ^ B;
```

$A \wedge B$ 只有在 B 是 A 的子集时才等价于集合差。否则它表示对称差，会把只属于 B 的元素也加入结果。

当使用 `long long` 存状态时，位移常量也要写成 `1LL << i`。

3 枚举一个集合的所有子集

枚举 mask 的所有非空子集：

```
for (int sub = mask; sub; sub = (sub - 1) & mask) {  
    // sub 是 mask 的一个非空子集  
}
```

它会从 `mask` 开始，按照数值递减的顺序访问每个非空子集，且不会产生集合外的位。

如果空集也需要参与转移，可以单独处理：

```
for (int sub = mask; ; sub = (sub - 1) & mask) {  
    // 使用 sub  
    if (sub == 0) break;  
}
```

3.1 枚举真子集

跳过集合本身，从第一个真子集开始：

```
for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {  
    // sub 是 mask 的非空真子集  
}
```

3.2 枚举补集

在枚举 `sub` 的同时，得到它相对于 `mask` 的补集：

```
for (int sub = mask; ; sub = (sub - 1) & mask) {  
    int rest = mask ^ sub; // sub 一定是 mask 的子集  
  
    if (sub == 0) break;  
}
```

这种写法常用于把一个集合拆成两个互不相交的部分。

4 枚举所有状态及其子集

```
for (int mask = 0; mask < (1 << n); ++mask) {
    for (int sub = mask; sub; sub = (sub - 1) & mask) {
        // 处理 (mask, sub)
    }
}
```

这段代码不是 $O(4^n)$ ，而是 $O(3^n)$ 。对每个元素来说，它只有三种归属：

1. 不在 `mask` 中；
2. 在 `mask` 中但不在 `sub` 中；
3. 同时在 `mask` 和 `sub` 中。

当 $n = 20$ 时， 3^n 通常已经过大，使用前必须结合常数和数据范围判断。

5 预处理合法状态

如果只有一部分状态合法，先把合法状态收集起来，往往能让后续转移更清晰。

例如，要求选择的元素不能相邻：

```
vector<int> valid;

for (int mask = 0; mask < (1 << n); ++mask) {
    if ((mask & (mask << 1)) == 0) {
        valid.push_back(mask);
    }
}
```

两行状态不能在同一列同时选择时，可以检查：

```
bool compatible(int a, int b) {
    return (a & b) == 0;
}
```

如果兼容关系会反复使用，可以预处理每个状态能够转移到哪些状态，避免在 DP 内层重复判断。

6 SOS DP

已知每个集合的权值 $f[\text{mask}]$ ，希望对每个集合求所有子集权值之和：

$$g[S] = \sum_{T \subseteq S} f[T]$$

可以使用 SOS DP：

```
vector<long long> g = f;

for (int bit = 0; bit < n; ++bit) {
    for (int mask = 0; mask < (1 << n); ++mask) {
        if (mask >> bit & 1) {
            g[mask] += g[mask ^ (1 << bit)];
        }
    }
}
```

总复杂度为 $O(n2^n)$ 。

如果要求所有超集的权值之和，则从不包含当前位的状态向包含当前位的状态转移：

```
vector<long long> g = f;

for (int bit = 0; bit < n; ++bit) {
    for (int mask = 0; mask < (1 << n); ++mask) {
        if ((mask >> bit & 1) == 0) {
            g[mask] += g[mask ^ (1 << bit)];
        }
    }
}
```

7 一个通用的子集 DP 骨架

下面的骨架表示：`dp[mask]` 是已经选择 `mask` 中元素时的最优值，每次加入一个尚未选择的元素。

```
const long long INF = 4e18;
const int total = 1 << n;
vector<long long> dp(total, INF);
dp[0] = 0;

for (int mask = 0; mask < total; ++mask) {
    if (dp[mask] == INF) continue;

    for (int i = 0; i < n; ++i) {
        if (mask >> i & 1) continue;

        int next = mask | (1 << i);
        dp[next] = min(dp[next], dp[mask] + cost(mask, i));
    }
}
```

具体题目中需要重新确认：

- `mask` 表示已经选择还是尚未选择；
- 转移顺序是否保证依赖的状态已经计算；
- 初始状态和答案状态分别是什么；
- 目标是最小值、最大值、计数还是可行性。

8 容易忘记的细节

- `1 << n` 使用的是 `int`；位数较多时改用 `1LL << n`。
- `&`、`|`、`^` 与比较运算混用时主动加括号。
- 数组大小通常是 `1 << n`，先估算内存。
- 子集枚举默认不包含空集，需要时单独处理。
- `A ^ B` 不是通用的集合差。
- 同一个无序划分可能被枚举两次，需要固定一个代表元素或除以二。
- 状态定义中不要保留能够从 `mask` 推导出的冗余维度。